

MAAGAP

Machine Analytics for Allocation, Governance and Assessment of Projects

INITIAL DATA AUDIT AND FEATURE ENGINEERING REPORT

SDLC Phase: Data Cleaning and Feature Engineering (Pre-Development)

Prepared for: Kirk Gamo

Reviewing Capacity: CS Thesis Panel (Data Audit) – Senior Data Scientist – Full-Stack AI Developer

Date: July 27, 2026

Source Manuscript: MAAGAP – A Machine Learning Framework for Predictive Risk Assessment and Optimized Resource Allocation in Government Project Management (Chapters 1–3)

Source Data: “Copy of 2022 conso Fund Transfer worksheet (2).xlsx” — PPDO Iloilo Province Consolidated Fund Transfer/Monitoring Workbook

1. Executive Summary

This report constitutes the initial Data Cleaning and Feature Engineering audit for the MAAGAP framework, conducted prior to any FastAPI or Next.js implementation, per the request scope. Chapters 1–3 of the manuscript were reviewed for their Data Collection and Preparation protocol (multiple imputation, IQR-based outlier detection, and temporal/categorical feature extraction), and this protocol was cross-referenced against a structural profile of the supplied raw data.

The central finding is that the supplied workbook is a compliance/disbursement monitoring ledger — not a project-lifecycle dataset. It records fund releases, liquidation status, and periodic field-monitoring remarks across nine heterogeneous sheets for the Iloilo Province PPDO, but it does not contain a stable project identifier, a scheduled/contracted completion date, or a validated project-type taxonomy. Each of these three fields is a mathematical prerequisite for the “Red Flag” slippage target defined in the Delimitation of the Study (Chapter 1) and for the sequential inputs required by the LSTM component (Chapter 3). Consequently, the target variable for the Level 1 meta-learner cannot currently be derived from this data alone, and none of the three external contextual sources required by Objective 1 (contractor records, PAGASA weather data, PSA economic indicators) are present.

This is not a disqualifying finding — it is the expected output of a first-pass audit on a manually maintained LGU spreadsheet, and it is explicitly anticipated by the manuscript's own “Data Availability” limitation (Chapter 1, line 154), which pre-authorizes synthetic data generation. Section 5 (Gap Analysis) and Section 6 (Preprocessing Pipeline) below detail exactly which fields must be engineered, imputed, or synthesized before model training can begin, and Section 2 flags two internal inconsistencies in the manuscript itself that should be resolved before the panel defense.

2. Manuscript Compliance Review

As a matter of academic rigor, two internal inconsistencies were identified between Chapter 1 and Chapter 3 that carry mathematical implications for the Level 1 meta-learner and should be reconciled prior to defense:

2.1 Risk Tier Cardinality Conflict (Three-Tier vs. Four-Tier)

1. Chapter 1, Delimitation of the Study (line 142): “Risk scoring is delimited to Low, Medium, and High categories to align with PPDO oversight protocols” — a three-tier scheme.
2. Chapter 3, System Architecture (line 688): Risk Scoring Engine classifies projects into “Low (0–0.3), Medium (0.3–0.7), and High (0.7–1.0)” — also three tiers, but with High spanning 0.7–1.0.
3. Chapter 3, Risk Classification Framework (line 617) and Objective 3: “Low (0–0.3), Medium (0.3–0.7), High (0.7–0.9), and Critical (0.9–1.0)” — four tiers.

The project brief (and Objective 1's Multinomial Logistic Regression meta-learner) presumes four discrete output classes. Since a Multinomial Logistic Regression's output cardinality is fixed at training time, this is

not a cosmetic wording issue — it determines the shape of the meta-learner's output layer and the labels used to train it. The panel is very likely to flag this contradiction. Recommend Chapter 1's Delimitation and Chapter 3's System Architecture paragraph be amended to state four tiers (Low, Medium, High, Critical) consistently with Objective 3 and the Risk Classification Framework, since the four-tier scheme is the more fully specified of the two and matches the stated meta-learner design.

2.2 Target Variable Definition Is Descriptive, Not Operational

Chapter 1 defines the predictive target qualitatively: a “Red Flag” is a project whose completion rate exceeds the standard duration (six months for non-infrastructure, one year for infrastructure) “without field extensions due to environmental or weather-related factors.” This is a sound conceptual definition, but Chapter 3's Data Collection and Preparation section does not specify the operational formula for it (i.e., which two date fields are subtracted, how a project is assigned to the infrastructure/non-infrastructure category, or how an approved weather-related extension is distinguished from an ordinary delay). Section 4.1 below shows that the raw data does not currently support this computation, so this gap should be resolved in the manuscript and the data pipeline simultaneously.

3. Data Dictionary and Health Check

3.1 File Inventory — Discrepancy Notice

The request specified two spreadsheet files (.xls and .xlsx). Only one native spreadsheet workbook was found in the project folder: “Copy of 2022 conso Fund Transfer worksheet (2).xlsx” (9 sheets, ~8.1 MB). The second file present, “...xlsx - MONITORING REPORT Con.csv”, is not an independent .xls file — it is a raw CSV export of a single sheet already contained in the .xlsx (“MONITORING REPORT Con”). Its presence is nonetheless diagnostically useful: comparing it to the source sheet exposes a real structural difference between how the same data is represented in each format (Finding DQ-1 below). Recommend confirming whether a genuinely separate .xls source (e.g., a different fiscal year, agency, or program) still needs to be supplied, since the current material represents a single consolidated workbook, not two independent datasets.

3.2 Workbook Structure

The workbook contains nine sheets of markedly different shape, grain, and purpose. Two (“Sheet1”, “Sheet2”) are pre-built pivot-table summaries, not raw records, and are excluded from feature engineering.

Sheet	Rows	Cols	Grain / Purpose
MONITORING REPORT Con	8,677	13	Field monitoring log, 2017–2026: one row per project visit (status, remarks, amount, dates).
Fund Transfer Con	21,376	23 (19 used)	Ledger of fund releases per barangay project, 2013–2024, with liquidation/monitoring status flags.
Liquidation Report Con	13,412	11 (10)	Liquidation submissions per project, linked loosely to Fund

Sheet	Rows	Cols	Grain / Purpose
		used)	Transfer Con by name/locality, not by key.
20% NTA Monitored	20	10	Small monitored subset funded by the 20% Development Fund ("NTA"), CY2022–2023.
SEF Monitored	86	12	Special Education Fund-financed projects monitored, mid-2023 snapshot.
FALGU, SBDP–LGSF Monitored	1	6	Single-record extract; effectively unusable at current volume.
monitored Projects geotagged	6	7 (of 26 declared)	Manually formatted PEO geotagging log; mostly merged header/footer cells.
Sheet1 / Sheet2	82 / 22	40 / 12	Pre-aggregated PivotTables (project counts by municipality/year) — summary only, not usable as model input rows.

3.3 Data Quality Findings

Findings are numbered DQ-1 through DQ-7 for traceability in later sections.

1. DQ-1 — Mixed date encodings within a single column. In “MONITORING REPORT Con”, the DATE RELEASED column mixes four distinct representations in the same field: Python/Excel datetime objects (672 cells), Excel serial integers (5,477 cells), literal 4-digit years typed as numbers, e.g. 2016 (part of the 307 float cells), and free-text strings such as “06/21/2023” (2,221 cells). The .xlsx and its own .csv export disagree on formatting for the same rows (e.g., ‘8/1/17’ in the CSV vs. a proper datetime in the source sheet), confirming the export pipeline itself is lossy. No date field can be trusted without per-cell type coercion.
2. DQ-2 — The completion date needed to compute delay is almost entirely absent, and is worst exactly where it matters most. “Date of Completion” is missing in 94.1% of MONITORING REPORT Con overall, 68.2% of 20% NTA Monitored, and 98.9% of SEF Monitored. Broken down by year, completeness collapses to 0% for 2023–2026 — the four most recent years, which is also the period the panel will most want to see modeled. Any delay/slippage target computed today would be trained almost entirely on 2018–2022 records.
3. DQ-3 — Categorical explosion from unstandardized free-text entry. STATUS in MONITORING REPORT Con contains 278 distinct raw strings (238 after case/whitespace normalization) that collapse to roughly 6–8 true categories — e.g., ‘Completed’, ‘Completed/ Fuctional’, ‘Completed/ Functional’, ‘Completed/ fuctional’, and ‘Completed/\nFuntional’ are all the same status with typos and stray whitespace/newlines. The same problem affects REMARKS (used as a liquidation-status proxy) and Municipality, where Fund Transfer Con alone shows 144 distinct municipality strings against roughly 42 real LGUs in Iloilo Province, and Liquidation Report Con shows 291. One-hot encoding directly on these columns today would create hundreds of spurious sparse features.
4. DQ-4 — No stable project identifier links the three transactional sheets. Fund Transfer Con's ‘code’ column, Liquidation Report Con's ‘Filename’/‘Ref. No.’, and MONITORING REPORT Con's ‘No.’ are each sheet-local sequence numbers, not shared keys — a spot check confirms they do not correspond to the same underlying values across sheets. Reconstructing a single project's lifecycle (fund released → liquidated → field-monitored) currently requires fuzzy matching on (Project Name, Barangay, Municipality, Year), which is itself an error-prone, unvalidated entity-resolution step.

5. DQ-5 — Numeric fields are stored as text and contain non-numeric sentinels. AMOUNT/Amount columns across sheets mix numeric cells with comma-formatted strings ('100,000.00') and the literal string 'NA' used as a missing-value marker rather than a blank cell. In MONITORING REPORT Con, 4.3% of AMOUNT cells fail numeric coercion outright.
6. DQ-6 — Cost distribution is extremely right-skewed, which will distort a naive IQR pass. Applying the Chapter 3-mandated IQR method directly to raw AMOUNT values (Q1=₱30,000, Q3=₱200,000, IQR=₱170,000) flags 12.9% of all records as outliers — too high a rate to be pure data-entry error, and it is driven by legitimate high-cost infrastructure projects sitting alongside low-cost non-infrastructure purchases (e.g., a single computer set) in the same column. IQR should be applied after either a log transform or stratification by inferred project category, not on the raw pooled distribution.
7. DQ-7 — Project type (infrastructure vs. non-infrastructure) is not a stored field and is not cleanly inferable from free text. A keyword-based classifier over NAME OF PROJECT (matching terms like 'road', 'construction', 'concreting' for infrastructure and 'purchase', 'training', 'equipment' for non-infrastructure) classifies only 40.2% of the 8,677 monitoring rows with confidence; 59.8% match neither dictionary or match both. Since project type determines which standard duration (6 months vs. 1 year) applies to the Chapter 1 Red Flag definition, this taxonomy gap directly blocks target-variable construction, not just feature engineering.

4. Feature Mapping to Level 0 Base Learners

4.1 Target Variable — Not Yet Derivable

No column in the workbook is an explicit delay/slippage/risk label. Per Chapter 1, the target must be engineered as: $\text{duration_actual} = \text{Date of Completion} - \text{Date Released}$ (or Date Monitored as a fallback start proxy), compared against a standard duration threshold (182 days for non-infrastructure, 365 days for infrastructure) to yield a binary Red Flag, which would then presumably be aggregated with a cost-overflow signal into the four-tier Low/Medium/High/Critical label. This computation is currently blocked by two of the findings above: DQ-2 (Date of Completion missing in 94–99% of rows, and 0% for 2023–2026) and DQ-7 (no reliable project-type taxonomy to select the correct threshold). Until both are resolved, no supervised target exists to train the Random Forest, XGBoost, or meta-learner against.

4.2 Static (Row-Level) Feature Candidates — for Random Forest & XGBoost

Candidate Feature	Source Column(s)	Status
Project cost	AMOUNT / Amount (all sheets)	Usable after numeric coercion + comma/'NA' cleanup (DQ-5).
Funding source	Source of Fund; Funding Source/Year	Usable after category normalization; sparse (54.9% missing in Fund Transfer Con).
Implementing level	FUNDS RELEASED TO: (Municipality/Barangay)	Usable after normalization; 2–3 clean categories.
Location (municipality/barangay)	LOCATION; Barangay; Municipality	Usable only after standardizing against an official PSGC municipality list (DQ-3).
Fiscal year	Year	Clean, numeric, near-zero missingness — one of the few reliable fields.
Project type (infra vs. non-infra)	NAME OF PROJECT (derived)	Not usable as-is; requires supervised text classification or manual coding (DQ-7).
Liquidation status	REMARKS / Status	Usable only after category normalization (DQ-3); currently 20+ near-duplicate labels.
Monitoring status	STATUS	Same as above — 278 raw values need mapping to ≈6 canonical classes.

4.3 Sequential Feature Candidates — for LSTM

Chapter 3 specifies LSTM inputs of cumulative expenditure, percentage completion, resource utilization, and issues per reporting interval. The workbook does contain three sheets that represent successive lifecycle stages of the same funding — Fund Transfer Con (release) → Liquidation Report Con (liquidation) → MONITORING REPORT Con (field status) — which is structurally the right shape for a time-indexed sequence per project. However, building that sequence today is blocked by DQ-4: there is no shared key

across the three sheets, so time-steps cannot be reliably assembled per project without a preceding, separately-validated record-linkage step (see Section 6, Step 3).

4.4 Level 0 Learner Input Summary

Base Learner	Expected Input Shape	Current Data Readiness
Random Forest	Static, tabular, mixed-type feature vector per project	Partially ready — numeric/categorical cleanup needed (Sec. 4.2); target variable missing (Sec. 4.1).
XGBoost	Same tabular feature vector as Random Forest	Same readiness as Random Forest.
LSTM	Ordered sequence of per-period observations per project	Not ready — requires entity resolution across three sheets before any sequence can be constructed (Sec. 4.3).
Meta-learner (Multinomial Logistic Regression)	Stacked Level 0 output probabilities + 4-class label	Blocked upstream by the target-variable gap; tier cardinality also needs manuscript reconciliation (Sec. 2.1).

5. Gap Analysis Against Chapter 1 Delimitation

Required Input (per Ch.1/Ch.3)	Present in Supplied Files?	Action Required
Historical project timelines (2016–2025)	Partially — 2013–2026 present, but completion dates absent post-2022	Proceed with 2017–2022 subset for initial modeling; treat 2023–2026 as a held-out gap to be backfilled.
Contractor performance records	Not present in any sheet	Not currently collectible from this workbook; confirm with PPDO whether a separate contractor/procurement log exists.
PAGASA weather data	Not present	External API integration has not begun; must be started in parallel, not after modeling, given typical lead time for historical climate data retrieval.
PSA economic indicators	Not present	Same as above — begin integration now.
Geographic variables (beyond municipality name)	Only municipality/barangay names, unstandardized	Join against an official PSGC/geospatial reference table once names are normalized.
Resource/inspector capacity constraints (for PuLP LP, Objective 4)	Not present	Out of scope for this dataset; needs a separate operational input from PPDO (5–6 inspectors, vehicle availability, etc., per Ch.1 delimitation).
Stable project identifier	Not present	Must be engineered via record linkage (Sec. 6, Step 3) before it can be treated as available.
Validated infrastructure/non-infrastructure taxonomy	Not present	Must be engineered via text classification or manual coding before target construction.

Direct answer to the scoping question: yes, synthetic data generation should begin now for two specific gaps — (a) contractor performance records, which appear nowhere in the supplied material, and (b) completion dates for 2023–2026, where real records are simply unrecorded rather than late. Both are explicitly pre-authorized by Chapter 1's "Data Availability" limitation. Separately, PAGASA and PSA integration should be scoped and started immediately in parallel with the cleaning work in Section 6, since external data acquisition (API access, historical archive requests) typically has its own lead time independent of internal preprocessing effort, and Objective 1 makes both sources a stated input to the meta-learner rather than an optional enrichment.

6. Proposed Preprocessing Pipeline (Python / Pandas)

The following sequence is proposed as the concrete implementation of Chapter 3's Data Collection and Preparation protocol, scoped specifically to this workbook, to be built as a standalone offline pipeline before it is wrapped by the FastAPI microservice.

Step 1 — Ingestion & sheet isolation

Load MONITORING REPORT Con, Fund Transfer Con, Liquidation Report Con, 20% NTA Monitored, and SEF Monitored with `openpyxl`, skipping Sheet1/Sheet2 (pivot summaries) and the near-empty FALGU/geotagged sheets. Preserve raw copies before any mutation.

Step 2 — Type coercion

Write per-column coercion functions: dates (handle Excel serial int, datetime object, and string formats explicitly, per DQ-1), currency (strip commas/‘Php’, map ‘NA’ to null, per DQ-5), and numeric IDs. Log a coercion-failure rate per column so silent data loss is visible.

Step 3 — Entity resolution

Construct a candidate project key via normalized (Project Name, Barangay, Municipality, Year of Fund Transfer) across Fund Transfer Con, Liquidation Report Con, and MONITORING REPORT Con, using fuzzy string matching (e.g., `rapidfuzz`) with a manually-reviewed confidence threshold, to address DQ-4. Persist the resulting crosswalk as its own artifact, since it is reusable and independently verifiable.

Step 4 — Categorical normalization

Lowercase, strip whitespace/newlines, and map STATUS, REMARKS, and Municipality to controlled vocabularies via an explicit lookup dictionary (not naive deduplication), addressing DQ-3. Municipality should additionally be validated against the official Iloilo Province PSGC list.

Step 5 — Project-type classification

Since keyword matching only resolves 40.2% of rows (DQ-7), either hand-label a stratified sample to train a lightweight text classifier (e.g., TF-IDF + logistic regression) over NAME OF PROJECT, or negotiate a manually-coded taxonomy field with PPDO going forward.

Step 6 — Target construction

Compute `duration_actual` from resolved start/completion dates; apply the 182-day / 365-day threshold from Step 5's category to derive the binary Red Flag; combine with a cost-overrun signal (post-normalization) into the four-tier Low/Medium/High/Critical label pending resolution of the Section 2.1 tier conflict. Rows with unresolvable completion dates (the 2023–2026 gap, DQ-2) should be excluded from supervised training and separately flagged for synthetic augmentation.

Step 7 — Missing-value imputation

Apply multiple imputation (e.g., `IterativeImputer` / MICE) per Chapter 3, fit separately on the resolved-entity, resolved-category subset from Steps 3–5 — imputing before entity resolution would propagate resolution errors into the imputed values.

Step 8 — Outlier detection

Apply the IQR method to log-transformed AMOUNT, stratified by the Step 5 project-type category, per DQ-6, rather than pooling infrastructure and non-infrastructure costs together.

Step 9 — Feature engineering

One-hot encode the now-normalized categoricals; extract month, quarter, and days-elapsed-since-fund-release from the coerced date fields; join PAGASA and PSA external feeds once available (Section 5).

Step 10 — Sequence assembly for LSTM

Using the Step 3 crosswalk, pivot the three transactional sheets into a per-project, per-period panel (release → liquidation → monitoring), forward-filling static attributes and leaving genuinely unreported periods as explicit gaps rather than zeros.

Step 11 — Normalization & split

Apply min-max scaling to numeric features per Chapter 3; perform the 70/30 train-test split at the project level (not the row level) to prevent the same project's multiple monitoring visits from leaking across the split.

7. Recommended Next Steps

- Resolve the two manuscript inconsistencies in Section 2 before the next panel review — these are the kind of internal contradiction a defense panel catches quickly.
- Confirm with the user/PPDO whether a genuine second workbook (the stated .xls) exists, since only one consolidated .xlsx was found (Section 3.1).
- Prioritize Steps 1–5 of the pipeline (Section 6) as the immediate engineering task — they are prerequisites for every downstream model, not just a cleanup nicety.
- Begin PAGASA and PSA integration scoping in parallel now, given their independent lead time (Section 5).
- Once the pipeline in Section 6 is implemented as Python modules, commit them following Conventional Commits on a feat/data-preprocessing branch (e.g., feat: implement date/currency type coercion, feat: add fuzzy entity resolution crosswalk, feat: derive red-flag target variable), and push after each logical milestone rather than as one large commit.
- No Next.js or FastAPI code has been written at this stage, consistent with the scope of this request; the FastAPI ML microservice should wrap the pipeline in Section 6 only after the target variable in Section 4.1 is confirmed derivable end-to-end on a validated sample.

8. Post-Implementation Data-Quality Addendum (2026-08-15)

This addendum documents a follow-up data-quality pass conducted by Claude Code against the live pipeline, prompted by a verification exercise for the Climate Data Coverage Gap known issue (Section 8.5 below). It extends, rather than revises, the original audit above (July 27, 2026): Sections 1-7 describe the raw workbook before any pipeline existed and remain an accurate historical record of that state. This section describes what the built pipeline's actual output looked like as of 2026-08-15, four new data-quality findings (continuing the DQ numbering from Section 3.3), and this Cowork session's independent verification of the implementation before any retraining or Supabase reseed proceeds.

8.1 New Findings (DQ-8 through DQ-11)

8. DQ-8 — Latent index-integrity bug in `mon_row_id`. The row identifier used to join monitoring rows back to the project crosswalk (`mon_row_id`) was derived via `reset_index()` after Steps 6-9 had already run. This only produced a correct result because no prior step had ever changed row count; any future row-drop would silently desynchronize it from the crosswalk's positional keys for every row after the first drop. Fixed by assigning `mon_row_id` once, at load time, before any filtering. This was a prerequisite fix, not optional — DQ-9/DQ-10/DQ-11 below all depend on it being correct.
9. DQ-9 — Un-floored study period admits data-entry-error dates. 3 labeled rows carried `D_start` values (2010-01-17, 2010-12-27, 2013-09-26) 8-13 years before every other row in their own monitoring batch. Chapter 1's stated "2016-2025" scope was tried first as a cutoff and rejected: it also silently swept up 381 legitimate 2015 rows with no evidence of error. The boundary actually used, `STUDY_PERIOD_START = 2015-01-01`, is grounded in a genuine gap in the raw data itself (zero rows anywhere in 2011, 2012, or 2014), which isolates exactly the 3 confirmed bad rows without touching the legitimate 2015 cohort. See second-brain D09-Study-Period-Floor.
10. DQ-10 — Directly-observed completion dates were never credibility-checked. Phase 6/8 already reject an implausible PROXY completion date (on or before `D_start`); no equivalent check existed for a DIRECTLY recorded Date of Completion. 133 rows (measured against the post-DQ-9/DQ-11 population) had a direct completion date on or before `D_start` — a non-positive duration that produced `RedFlag=0` by mechanical construction, not observed evidence. These are now discarded and re-routed through the existing proxy-recovery machinery: 43 recovered a credible proxy and remain labeled; the rest correctly fall through to unresolved. See second-brain D10-Direct-Date-Credibility-Check.
11. DQ-11 — Exact-duplicate ledger rows. 503 of the full 8,784-row monitoring population share an identical (project name, date released, date monitored, amount) fingerprint with another row carrying a DIFFERENT `mon_row_id` — confirmed genuine duplicate entries in the source spreadsheet, not a crosswalk/join artifact. Deduplicated (`keep="first"`) across the full pipeline, train/test and inference alike. Known, called-out exception: LSTM sequence assembly still includes the duplicate raw events, since deduping ahead of it safely would require separate changes to its position-fragile crosswalk dependency. See second-brain D11-Exact-Duplicate-Removal.

8.2 Verified Net Effect

Cross-checked two independent ways — the train+test row-count sum, and the pipeline's own logged checkpoint after Step 6 — and both agree with the numbers below.

- **Full population.** Full monitoring population: 8,784 → 8,278 rows (503 DQ-11 duplicates + 3 DQ-9 study-period rows dropped, full pipeline scope).
- **Labeled population.** 5,159 → 4,804 rows (train 3,363 / test 1,441). Prior headline metrics in Section 2 of the methodology report and HANDOFF.md Section 2 describe the PRE-cleanup 5,159-row population and are marked stale pending retraining.

8.3 Independent Verification Notes (this Cowork session)

Rather than accept the implementation report at face value, the actual diff to `feature_engineering.py` was read against the three new decision records. The DQ-8 fix (`mon_row_id` moved to load time) is implemented exactly as described. Phase order in code is confirmed as DQ-11 (dedup) → DQ-9 (study

floor) → DQ-10 (credibility check) within `construct_target_variable()`, and the 8,278 full-population checkpoint ($8,784 - 503 - 3$) matches this order exactly.

Open item — DQ-10 bookkeeping gap (not yet resolved): the decision record states 133 rows flagged non-credible, 43 recovered, implying 90 should become newly unresolved — but the final population accounting ($5,159 - 285 - 3 - 67 = 4,804$) attributes only 67 to DQ-10, a 23-row gap. The most likely explanation, consistent with how the check is written (it runs against the full 8,278-row post-DQ-9/DQ-11 population, not just the previously-labeled subset): roughly 23 of the 133 flagged rows were never part of the labeled 5,159 to begin with — for example, rows with an Unclassified project type, which per the existing Known-Issues entry can never be labeled regardless of date credibility, since `T_standard` is undefined for them. This is plausible but NOT yet explicitly confirmed against the actual data. Recommend a one-line confirmation query before this figure is cited in the manuscript.

Open item — work is currently uncommitted: as of this verification, `feature_engineering.py`, `HANDOFF.md`, and the climate-gap known-issue note are modified-but-unstaged on the main branch (three new decision records are untracked), one commit ahead of origin. Per this project's own Git Conventions, a completed and verified milestone like this should be committed — ideally as atomic commits separating the DQ-8 index fix, DQ-9 floor, DQ-10 credibility check, and DQ-11 dedup — before Phase B (retraining) proceeds, so the pre-cleanup and post-cleanup pipeline states remain independently reproducible from git history.

8.4 Recommendation: Phase B (Retraining + Supabase Reseed)

Conditional go. The headline population numbers (4,804 labeled / 8,278 full) are corroborated two independent ways and are trustworthy; neither open item above indicates a defect in the pipeline's actual output, only in how completely that output's derivation is documented. Recommend, in order:

- Commit the Phase A changes (DQ-8 through DQ-11) as atomic, conventional commits.
- Confirm the DQ-10 bookkeeping gap with a direct query, and tighten DQ5's wording accordingly.
- Proceed to retrain RF/XGBoost/LSTM/meta-learner and reseed Supabase (`HANDOFF.md` Section 4, steps 4-7), then update the methodology report's headline metrics and `HANDOFF.md` Section 2's staleness flag.

8.5 Cross-Reference: Climate Data Coverage Gap

This cleanup pass was prompted by quantifying the Climate Data Coverage Gap known issue (the PAGASA climate-data request is capped at 31 December 2024 by PAGASA's own form). Recomputed against the post-cleanup 4,804-row population: 189 rows (3.93%) fall in 2025, evenly split across train (3.75%) and test (4.37%) — still footnote-scale, not a design concern. See `second-brain/05-Known-Issues/Issue-Climate-Data-Coverage-Gap.md` for the full detail.

9. Currency Check — Status of Sections 1–7 Findings (2026-08-15)

Sections 1–7 were written July 27, 2026, before any pipeline existed, and describe the raw workbook and a PROPOSED plan. Three weeks of implementation since (Section 8 above, plus the fuller history in `HANDOFF.md`'s Recent Major Fixes) have resolved, superseded, or partially addressed most of those findings. This section is a claim-by-claim status check, verified against the current codebase during this Cowork session, so that a reader does not mistake Sections 1–7's “not yet derivable” / “proposed” language for the system's current state.

9.1 Resolved

- **Executive Summary / Section 4.1 (“target variable cannot currently be derived”).** the target variable is now fully operational: RedFlag is derived end-to-end by `construct_target_variable()` across Phases 1–11, documented in decision records D01–D03 and D09–D11 and the methodology report. Every headline metric in HANDOFF.md Section 2 trains against it.
- **DQ-1 (mixed date encodings).** `coerce_date_column()` in `preprocess.py` explicitly handles Excel serial integers, datetime objects, and free-text formats per-cell.
- **DQ-3, STATUS half (categorical explosion).** `normalize_categorical()` maps raw STATUS strings to a controlled vocabulary (STATUS_clean) via an explicit lookup, exactly as Section 6 Step 4 proposed.
- **DQ-4 (no stable project identifier).** `build_project_crosswalk()` implements Section 6's proposed Step 3 (fuzzy matching on Project Name / Barangay / Municipality / Year), later hardened by the barangay veto (D04-Barangay-Veto-Crosswalk).
- **DQ-5 (numeric fields stored as text).** `coerce_currency_column()` in `preprocess.py` strips commas/“Php” and maps “NA” sentinels to null.
- **DQ-6 (naive IQR on skewed cost distribution).** `flag_outliers_iqr_stratified()` applies IQR stratified by `project_type`, exactly as Section 6 Step 8 proposed.
- **Section 4.3 (LSTM sequencing blocked by DQ-4).** `assemble_lstm_sequences()` now runs against the crosswalk; the LSTM base learner trains end-to-end (HANDOFF Section 2: n=1,124 train / 477 test).
- **Section 7 (“No Next.js or FastAPI code has been written...”).** both now exist and are substantially built: FastAPI (`ml-service/main.py`, six live endpoints including live-score and model-metrics) and a full Next.js App Router frontend (Tremor dashboards, SHAP bar charts, PPA/Schedule/Inspector views).
- **Section 7 (“Prioritize Steps 1–5”).** implemented across the phases in `preprocess.py/feature_engineering.py`.
- **Section 7 (Conventional Commits, push per milestone).** the project's git history matches this pattern (see HANDOFF.md Section 3).
- **Section 5 (synthetic contractor data generation).** `generate_synthetic_data.py` produces a 150-row contractor table, wired into the feature set (`contractor_spec_*`, `historical_delay_rate`, `reliability_score`) — see 9.2 for the caveat this leaves.

9.2 Partially addressed — improved, but not closed the way Section 6 proposed

- **DQ-7 (project-type classification).** `classify_project_type()`'s own docstring in `preprocess.py` calls itself “an improved heuristic (v2), not a final classifier” and explicitly still points back to this report's Section 6 Step 5 recommendation (a supervised TF-IDF + logistic-regression classifier on a hand-labeled sample) as not-yet-done. Coverage improved substantially — from the original 40.2% to roughly 80.9% (100% minus the current 19.1% Unclassified rate) — via a better keyword dictionary, not the ML classifier this report recommended. The residual 19.1% is now tracked as the Unclassified-Project-Type Known Issue rather than a blocker.
- **Section 5 (PAGASA integration).** a PAGASA climate-data request was submitted 2026-08-15 (see `second-brain/05-Known-Issues/Issue-Climate-Data-Coverage-Gap.md`) — scoping has begun as this report originally recommended, but real weather data has not yet landed; `is_wet_season_release` remains a release-month proxy meanwhile.

- **Contractor data.** the gap this report identified (no contractor records in the supplied material) was closed with synthetic placeholder data, exactly as Section 5 pre-authorized — but this remains a declared Chapter 1 Data Availability limitation, not real PhilGEPS/PPDO contractor-performance data.

9.3 Still accurate as written / unresolved

- **PSA economic-indicator integration.** optimization_engine.py's own source comment describes fetch_psa_data.py as “a clearly-flagged placeholder for a domain expert to” complete — this gap is exactly as open as it was on July 27.
- **DQ-2 (completion date missing 94%+).** the raw finding is still an accurate description of the source workbook — the missing rate in the raw Date of Completion column hasn't changed and can't. What changed is the CONCLUSION drawn from it in Section 4.1: the proxy/credibility-check machinery (D01–D03, D09–D10) now routes around this gap for the labeled population rather than being blocked by it. Read DQ-2 as still-true background, not as “target variable is blocked.”
- **Barangay-level canonicalization.** still an open Known Issue after the D04 veto fix reduced but didn't eliminate cross-barangay conflation risk.

9.4 Unverified from this repo alone

- **Sections 2.1 / 2.2 (manuscript tier conflict / operational definition).** the SYSTEM resolved this in practice — every trained model outputs four discrete tiers (Low/Medium/High/Critical; see HANDOFF Section 2) and the operational RedFlag formula is fully specified in code and the decision records. Whether Chapters 1 and 3 of the manuscript DOCUMENT itself were actually edited to remove the textual inconsistency could not be checked — no manuscript/chapter files exist in this repository to inspect. Confirm directly against the manuscript before defense.
- **Section 3.1 (second .xls workbook).** the pipeline appears to have proceeded on the single consolidated .xlsx alone (no second raw source file referenced anywhere in preprocess.py or HANDOFF.md), suggesting this was resolved as “no second file exists” — but that was not explicitly confirmed in any document read for this check.

9.5 Correction to Section 8 (this same addendum)

Section 8.1–8.3 above originally cited the Phase 9/10/11 decision records as D04/D05/D06 — correct at the moment they were written earlier the same day (2026-08-15), but those files were renamed to D09/D10/D11 shortly after, to resolve a numbering collision with the pre-existing D04-Barangay-Veto-Crosswalk / D05-Refunded-Status / D06-Status-Excludes-Scheduling records. All three references in Section 8 have been corrected to D09/D10/D11 to match the current second-brain state.